

LAB # 10

FOR LOOP, CONTROL STRUCTURE IN MIPS

OBJECTIVE

To implement translation of “for loop” control structures in MIPS assembly language.

THEORY

Translation of a “FOR LOOP” Control Structure

Obviously a “ for loop ” control structure is very useful. Let us suppose that a programmer initially developed an algorithm containing the following pseudocode. In one sentence, can you describe what this algorithm accomplishes?

```
int a0 = 0;
for(int t0 = 10; t0 > 0; t0--) {
    a0 = a0 + t0;
}
```

The following is a translation of the above “for-loop” pseudocode to MIPS assembly language code.

```
li $a0, 0      # $a0 = 0 li $t0, 10    # Initialize loop counter to 10 loop:
add $a0, $a0, $t0 addi $t0, $t0, -1    #
Decrement loop counter bgtz $t0, loop    # If
($t0 > 0) Branch to loop
```

BRANCH IF GREATER THAN ZERO

```
bgtz $t0, loop # branch to loop if $t0 > 0
```

PROGRAM#1:

Write a program in MIPS assembly language that translate FOR LOOP control structure.

```
##### DATA SEGMENT #####
.data
counter: .asciiz "\nValue of counter ($t0): "
total: .asciiz "\tValue of sum ($a2): "
newline: .asciiz "\n"

##### TEXT SEGMENT #####
```

```

.text
.globl main

main:
    # Initialization
    li $a2, 0      # $a2 = 0 (sum)
    li $t0, 10     # $t0 = 10 (loop counter)

loop:
    add $a2, $a2, $t0 # sum = sum + counter

    # Print counter message
    la $a0, counter
    li $v0, 4
    syscall

    # Print counter value
    move $a0, $t0
    li $v0, 1
    syscall

    # Print total message
    la $a0, total
    li $v0, 4
    syscall

    # Print sum value
    move $a0, $a2
    li $v0, 1
    syscall

    # Print new line
    la $a0, newline
    li $v0, 4
    syscall

    # Decrement loop counter
    addi $t0, $t0, -1

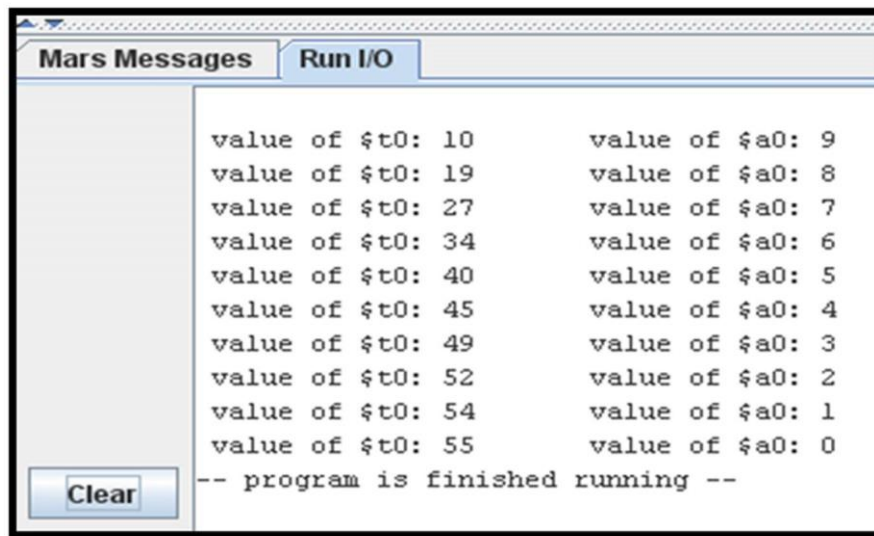
    # Loop condition
    bgtz $t0, loop  # if ($t0 > 0) repeat loop

end:

```

```
li $v0, 10    # Exit program
syscall
```

OUTPUT:



```
value of count, $t0: 10| value of sum, $a0: 10
value of count, $t0: 9  value of sum, $a0: 19
value of count, $t0: 8  value of sum, $a0: 27
value of count, $t0: 7  value of sum, $a0: 34
value of count, $t0: 6  value of sum, $a0: 40
value of count, $t0: 5  value of sum, $a0: 45
value of count, $t0: 4  value of sum, $a0: 49
value of count, $t0: 3  value of sum, $a0: 52
value of count, $t0: 2  value of sum, $a0: 54
value of count, $t0: 1  value of sum, $a0: 55
-- program is finished running --
```

LAB TASK

1. Write a program in MIPS assembly language that take input and display whether number is prime or not?
2. Write a program in MIPS assembler that takes a group of 4 integers as input, calculates their sum, and prints the calculated sum along with the message "Even" if the sum is even, or "The sum of integers is odd" if the sum is odd.